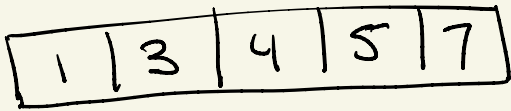
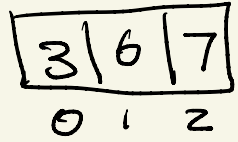
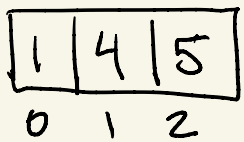
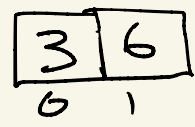
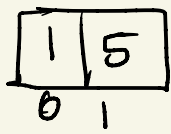
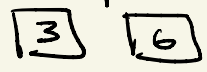
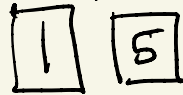
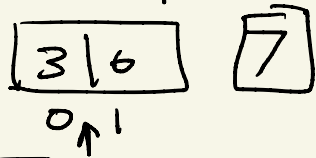
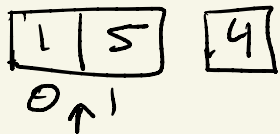
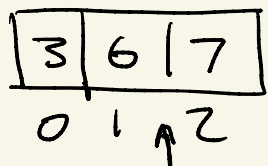
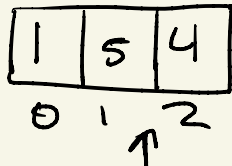
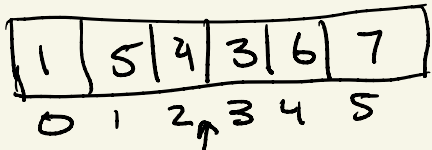


1. MERGE SORT



2. INSERTION SORT

- sorted

-21	5	7	-10	61	8	3	10
-----	---	---	-----	----	---	---	----

-21	5	7	-10	61	8	3	10
-----	---	---	-----	----	---	---	----

-21	5	7	-10	61	8	3	10
-----	---	---	-----	----	---	---	----

-21	-10	5	7	61	8	3	10
-----	-----	---	---	----	---	---	----

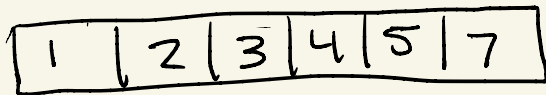
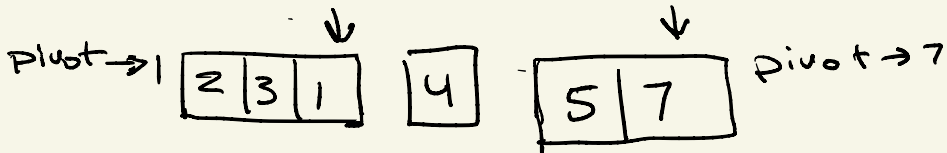
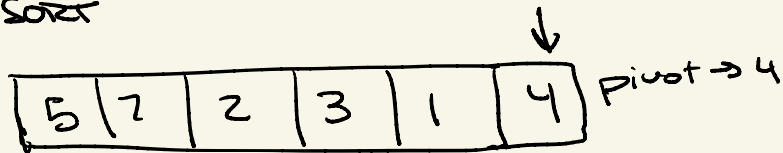
-21	-10	5	7	61	8	3	10
-----	-----	---	---	----	---	---	----

-21	-10	5	7	8	61	3	10
-----	-----	---	---	---	----	---	----

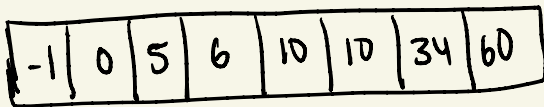
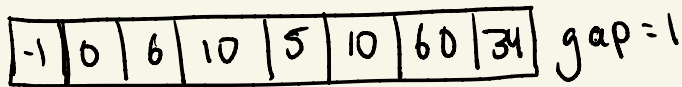
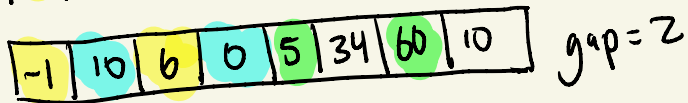
-21	-10	3	5	7	8	61	10
-----	-----	---	---	---	---	----	----

-21	-10	3	5	7	8	10	61
-----	-----	---	---	---	---	----	----

3. QUICK SORT



1. SHELL SORT



5. RANKING SORTING ALGORITHM

3. Bubble - $O(n^2)$

n is equal to the number of inputs. It has a time complexity of $O(n^2)$ because every time it loops it has to look through every element in the array & swap. This the simplest algorithm so despite it having the same time complexity, it is probably slower in some cases.

2. INSERTION/SELECTION SORT - $O(n^2)$

n is equal to the number of inputs. Both insertion & selection sort are very similar. Both require you to look through the entire array for every input. However both of these are more complicated than bubble sort so, may work better in certain circumstances.

1. SHELL/QUICK/MERGE - $O(n \log n)$

All three of these algorithms break up the array into smaller parts. So they have to scan through the entire array less times than the simpler algorithms. I believe that they will be much more efficient than the previous algorithms.